

35.1. Perso — dessiner des personnages

`#include <Perso>` : Perso NOM = { ... }; un personnage (16 × 16 ou 32 × 32). Deux personnages, deux dessins, DEUX FICHIERS : `perso_BONHOMME.cpp` et `perso_ROBOT.cpp`, rangés par une liste, `personnages.cpp`, qui porte UNE fois `#include <Perso>` ; la source verse la liste d'une ligne et les pose pour les voir.

1. Ce qu'il faut comprendre

Ce que c'est : un Perso est un personnage : un dessin de 16 × 16 pixels (seize rangées de seize signes) ou de 32 × 32 (trente-deux rangées de trente-deux). Sa taille se lit sur son dessin. Le compilateur le coupe en tuiles de 8 × 8 qui se suivent : quatre pour un 16 × 16 — haut-gauche, haut-droite, bas-gauche, bas-droite.

Un personnage, un fichier : le dessin 1, le dessin 2. BONHOMME est dessiné dans `perso_BONHOMME.cpp`, ROBOT dans `perso_ROBOT.cpp` : deux onglets à côté de `principal.cpp`, un dessin chacun — comme deux images. Pour changer ROBOT, on n'ouvre que son fichier.

Deux personnages, deux noms : BONHOMME et ROBOT. Chacun a son `Perso NOM = { ... };`. Le nom est ce qui compte : c'est par lui qu'on l'appellera partout dans le jeu.

La liste, `personnages.cpp` : l'include qui inclut tous les fichiers. Elle porte `#include <Perso>` UNE fois, pour tous les personnages (cette ligne n'est pas celle d'un personnage : elle dit « on a le droit d'écrire des Perso »), puis une ligne par personnage : `#include "perso_BONHOMME.cpp"`, `#include "perso_ROBOT.cpp"`. `principal.cpp` n'a qu'UNE ligne pour les avoir tous : `#include "personnages.cpp"`.

Comment ça colle : `#include "personnages.cpp"` colle la liste ; la liste colle chaque `perso_...` (des guillemets : des fichiers à toi, comme `variables.h` au 0.67). BONHOMME et ROBOT arrivent, chacun avec son nom.

Pour les voir tout de suite, on les pose sur le fond, chacun en quatre cases, avec `poser()` que tu connais : `BONHOMME` est le numéro de sa première tuile (le quart haut-gauche), `BONHOMME + 1` la suivante (haut-droite), `BONHOMME + 2` (bas-gauche), `BONHOMME + 3` (bas-droite). Pareil pour `ROBOT`.

Déroulé : BONHOMME à gauche, en colonnes 4 et 5, lignes 8 et 9 ; ROBOT à droite, en colonnes 14 et 15, lignes 8 et 9. Ses tuiles viennent juste après celles de BONHOMME : `ROBOT` vaut `BONHOMME + 4`.

Sur le fond, ils ne bougent pas : ils sont collés à la grille des cases. L'étape suivante, `sprite16()`, en fait des lutins, qui se déplacent au pixel près.

Ce qu'ils coûtent : 64 octets de dessin chacun (quatre tuiles). La ligne `#include <Perso>`, elle, ne coûte rien.

Essaie : dans l'onglet `perso_ROBOT.cpp`, change les yeux de ROBOT (les `+`) : seul ROBOT change. Ou ajoute un troisième personnage : un fichier `perso_...` de plus, et une ligne de plus dans la liste `personnages.cpp` — `principal.cpp` ne change pas.

On retrouve Perso dans 66 leçons

[35.6](#), [35.7](#), [35.8](#), [35.9](#), [35.10](#), [35.12](#), [35.16](#), [35.17](#), [35.18](#), [35.19](#), [36](#), [47](#), [2.01](#), [2.02](#), [2.03](#), [2.04](#), [2.05](#), [2.06](#), [2.07](#), [2.08](#), [2.09](#), [2.10](#), [2.11](#), [2.12](#), [2.13](#), [2.14](#), [2.15](#), [2.16](#), [2.17](#), [2.18](#), [2.19](#), [2.20](#), [2.21](#), [2.22](#), [2.23](#), [2.24](#), [2.25](#), [2.26](#), [2.27](#), [2.28](#), [2.29](#), [2.30](#), [2.31](#), [2.32](#), [2.33](#), [2.34](#), [2.36](#), [2.37](#), [2.38](#), [3.01](#), [3.02](#), [3.03](#), [3.04](#), [3.05](#), [3.06](#), [3.07](#), [3.08](#), [3.09](#), [3.10](#), [3.11](#), [3.12](#), [3.13](#), [3.14](#), [3.15](#), [3.16](#), [3.17](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **329 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	– la mise en route de la console
main()	55 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
CopierTuiles	31 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
Tuiles	128 o	← CopierTuiles	– 0 lettre sur 43, et 8 tuiles dessinées

Lettres gravées : aucune

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser() × 8	48 o	sur place, dans main()
image() × 1	3 o	sur place, dans main() + AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <Perso> : un personnage de 16 × 16 ----
// BONHOMME et ROBOT sont dessinés chacun dans SON fichier ; la liste
// personnages.cpp les inclut tous. Ici, UNE ligne verse la liste, et on
// pose chacun en quatre cases pour les voir.

#include <poser>           // poser() : une tuile sur une case du fond
#include "personnages.cpp" // la liste : BONHOMME et ROBOT
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Un Perso de 16 × 16, ce sont quatre tuiles qui se suivent :
    //   NOM      le quart haut-gauche    NOM + 1  le quart haut-droite
    //   NOM + 2  le quart bas-gauche     NOM + 3  le quart bas-droite

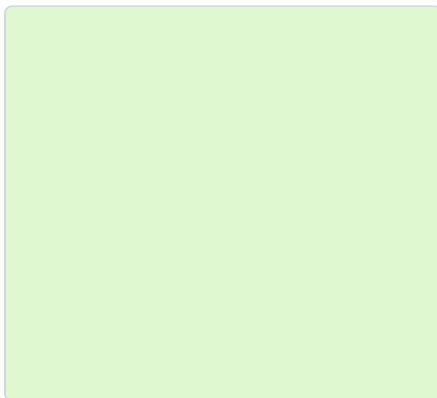
    // BONHOMME, à gauche : colonnes 4 et 5, lignes 8 et 9
    poser(4, 8, BONHOMME);
    poser(5, 8, BONHOMME + 1);
    poser(4, 9, BONHOMME + 2);
    poser(5, 9, BONHOMME + 3);

    // ROBOT, à droite : colonnes 14 et 15, lignes 8 et 9
    poser(14, 8, ROBOT);
    poser(15, 8, ROBOT + 1);
    poser(14, 9, ROBOT + 2);
    poser(15, 9, ROBOT + 3);

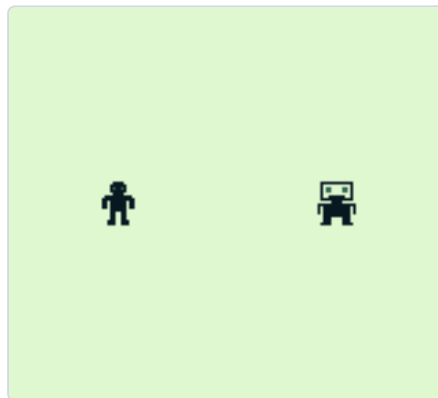
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Deux personnages sur le fond : le bonhomme à gauche, le robot à droite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ BONHOMME : ses quatre quarts en carré, colonnes 4-5 (tuile 44)

✓ ROBOT : ses quatre quarts en carré, colonnes 14-15

✓ ROBOT a ses tuiles à lui, juste après BONHOMME (44, 48)

✓ et on les voit : la tête du bonhomme, le haut du robot

Ensuite : **35.2. La fonction `sprite16()` — un personnage de 16×16 —** chapitre 6 : Dessiner. · [gameboy3](#) — écrire une cartouche Game Boy en C++